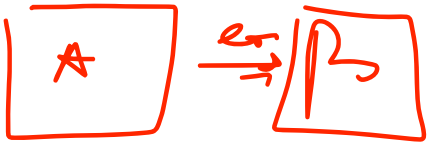
THỰC HÀNH 3: KẾ THỪA & ĐA HÌNH (INHERITANCE & POLYMORPHISM)

🎯 MỤC TIÊU BÀI HỌC



Sau khi hoàn thành bài thực hành này, bạn sẽ:

1. ✅ **Sử dụng kế thừa** (`extends`) để tái sử dụng code
2. ✅ **Hiểu Overriding** và khi nào cần dùng `@Override`
3. ✅ **Phân biệt** Overriding vs Overloading
4. ✅ **Sử dụng Abstract Class** và Interface đúng cách
5. ✅ **Áp dụng Polymorphism** để loại bỏ if/else
6. ✅ **Hiểu Upcasting/Downcasting** và `instanceof`

⏱️ **Thời gian ước tính:** 6-8 giờ

📋 **Kiến thức cần có trước:**

- Đã hoàn thành Thực hành 2
- Đã đọc Chương 3: Inheritance & Polymorphism
- Hiểu về Class, Constructor, Method

🧪 PHẦN 1: HƯỚNG DẪN CHI TIẾT (STEP-BY-STEP)

LAB 3.1: Vườn Thú Ồn Ào (Polymorphism Demo)

Mục tiêu: Hiểu cơ chế Runtime Dynamic Binding

Bước 1: Tạo Class Cha (Animal)

Tạo file: `Animal.java` trong package `com.bootcamp.lab3`

```java
package com.bootcamp.lab3;

public class Animal {
    protected String name;  // protected để subclass truy cập được

    public Animal(String name) {
        this.name = name;
    }

    // Method chung - có thể bị override
    public void makeSound() {
        System.out.println(name + " phát ra một âm thanh lạ...");
    }
```

```
    public void eat() {
        System.out.println(name + " đang ăn...");
    }

    public void displayInfo() {
        System.out.println("Động vật: " + name);
    }
}
```

Giải thích:

- `protected`: Subclass có thể truy cập (sẽ học ở Chương 3)
- Method `makeSound()`: Có thể bị override ở subclass

Bước 2: Tạo Class Con (Dog)

Tạo file: `Dog.java`

```
package com.bootcamp.lab3;

public class Dog extends Animal {
    private String breed;

    // Constructor phải gọi super()
    public Dog(String name, String breed) {
        super(name);  // Gọi constructor của Animal
        this.breed = breed;
    }

    // OVERRIDE: Ghi đè method của cha
    @Override
    public void makeSound() {
        System.out.println("🐕 " + name + ": Gâu gâu!");
    }

    // Method riêng của Dog
    public void fetch() {
        System.out.println(name + " đang nhặt bóng về.");
    }

    @Override
    public void displayInfo() {
        super.displayInfo();  // Gọi method của cha
        System.out.println("Giống: " + breed);
    }
}
```

Giải thích:

1. `extends Animal`:

    - `Dog` kế thừa từ `Animal`
    - Có tất cả attributes và methods của `Animal`
    - Quan hệ "IS-A": Dog LÀ MỘT Animal

2. **`super(name)`**:
    - Gọi constructor của class cha
    - Phải là dòng đầu tiên trong constructor
    - Khởi tạo phần kế thừa trước

3. **`@Override`**:
    - Annotation đánh dấu đây là override
    - Compiler kiểm tra: Đảm bảo đúng là override
    - Nếu superclass thay đổi tên method → Báo lỗi ngay

4. **`super.displayInfo()`**:
    - Gọi method của class cha
    - Dùng khi muốn mở rộng, không thay thế hoàn toàn

Bước 3: Tạo Class Con (Cat)

Tạo file: `Cat.java`

```
package com.bootcamp.lab3;

public class Cat extends Animal {
    public Cat(String name) {
        super(name);
    }

    @Override
    public void makeSound() {
        System.out.println("🐈 " + name + ": Meow meow!");
    }

    // Method riêng của Cat
    public void scratch() {
        System.out.println(name + " đang cào cào...");
    }
}
```

Bước 4: Test Polymorphism

Tạo file: `ZooApp.java`

```
package com.bootcamp.lab3;
```

```java
public class ZooApp {
    public static void main(String[] args) {
        System.out.println("=== VƯỜN THÚ ỒN ÀO ===\n");

        // Tạo mảng chứa Animal (kiểu cha)
        Animal[] zoo = new Animal[3];

        // Upcasting: Con -> Cha (Tự động)
        zoo[0] = new Dog("LuLu", "Golden Retriever");
        zoo[1] = new Cat("MiMi");
        zoo[2] = new Dog("KiKi", "Husky");

        System.out.println("=== GIỜ ĂN ĐẾN RỒI ===\n");

        // POLYMORPHISM: Cùng một method call, nhưng hành vi khác nhau
        for (Animal animal : zoo) {
            animal.makeSound();  // Gọi method của object thực tế (Dog
hoặc Cat)
        }

        System.out.println("\n=== CHI TIẾT TỪNG ĐỘNG VẬT ===\n");

        for (Animal animal : zoo) {
            animal.displayInfo();

            // Downcasting: Ép về kiểu con để gọi method riêng
            if (animal instanceof Dog) {
                Dog dog = (Dog) animal;  // Ép kiểu
                dog.fetch();
            } else if (animal instanceof Cat) {
                Cat cat = (Cat) animal;
                cat.scratch();
            }
            System.out.println();
        }
    }
}
```

Kết quả:

```
=== VƯỜN THÚ ỒN ÀO ===

=== GIỜ ĂN ĐẾN RỒI ===

🐕 LuLu: Gâu gâu!
🐈 MiMi: Meow meow!
🐕 KiKi: Gâu gâu!

=== CHI TIẾT TỪNG ĐỘNG VẬT ===

Động vật: LuLu
Giống: Golden Retriever
```

```
LuLu đang nhặt bóng về.
...
```

Giải thích:

1. **Polymorphism:**
   - `animal.makeSound()`: Cùng một dòng code
   - Nhưng gọi method của object thực tế (Dog hoặc Cat)
   - Đây là **Runtime Dynamic Binding**
2. **Upcasting:**
   - `Animal animal = new Dog(...)`: Tự động
   - Biến kiểu `Animal` nhưng chứa object `Dog`
3. **Downcasting:**
   - `Dog dog = (Dog) animal`: Phải ép kiểu
   - Dùng `instanceof` để kiểm tra an toàn
   - Cho phép gọi method riêng của subclass

LAB 3.2: Thanh Toán Lương (Abstract Class)

Mục tiêu: Hiểu Abstract Class và Template Method Pattern

Bước 1: Tạo Abstract Class

Tạo file: `Employee.java`

```java
package com.bootcamp.lab3;

public abstract class Employee {
    protected String name;
    protected String id;

    public Employee(String name, String id) {
        this.name = name;
        this.id = id;
    }

    // ABSTRACT METHOD: Chỉ có tên, không có body
    // Subclass BẮT BUỘC phải implement
    public abstract double calculateSalary();

    // CONCRETE METHOD: Đã có implementation
    public void display() {
        System.out.printf("NV: %-15s | ID: %-10s | Lương: %.0f VND\n",
                          name, id, calculateSalary());
        // Gọi abstract method được! Vì khi chạy, subclass đã implement
```

```
rồi
    }

    // GETTERS
    public String getName() { return name; }
    public String getId() { return id; }
}
```

Giải thích:

- `abstract class`: Không thể tạo object trực tiếp
- `abstract method`: Chỉ định nghĩa, không implement
- Subclass phải implement tất cả abstract methods

Bước 2: Tạo Concrete Classes

Tạo file: `Manager.java`

```
package com.bootcamp.lab3;

public class Manager extends Employee {
    private static final double BASE_SALARY = 20000000;  // Lương cứng 20
triệu

    public Manager(String name, String id) {
        super(name, id);
    }

    @Override
    public double calculateSalary() {
        return BASE_SALARY;
    }
}
```

Tạo file: `Worker.java`

```
package com.bootcamp.lab3;

public class Worker extends Employee {
    private int hoursWorked;
    private static final double HOURLY_RATE = 50000;  // 50k/giờ

    public Worker(String name, String id, int hoursWorked) {
        super(name, id);
        this.hoursWorked = hoursWorked;
    }

    @Override
    public double calculateSalary() {
```

```
        return hoursWorked * HOURLY_RATE;
    }

    public int getHoursWorked() {
        return hoursWorked;
    }
}
```

Bước 3: Test Abstract Class

Tạo file: `PayrollApp.java`

```
package com.bootcamp.lab3;

public class PayrollApp {
    public static void main(String[] args) {
        System.out.println("=== BẢNG LƯƠNG NHÂN VIÊN ===\n");

        // Tạo mảng Employee (kiểu cha)
        Employee[] employees = new Employee[4];

        employees[0] = new Manager("Nguyễn Văn A", "M001");
        employees[1] = new Worker("Trần Thị B", "W001", 160);
        employees[2] = new Manager("Lê Văn C", "M002");
        employees[3] = new Worker("Phạm Thị D", "W002", 200);

        // POLYMORPHISM: Tính lương tự động theo loại nhân viên
        double totalSalary = 0;
        for (Employee emp : employees) {
            emp.display();
            totalSalary += emp.calculateSalary();
        }

        System.out.println("\n=== TỔNG LƯƠNG ====");
        System.out.printf("Tổng lương: %.0f VND\n", totalSalary);
    }
}
```

Giải thích:

- `Employee[]`: Mảng kiểu cha
- Mỗi object tự biết cách tính lương của mình
- Không cần if/else để phân biệt Manager hay Worker

LAB 3.3: Interface - IPlayable

Mục tiêu: Hiểu Interface và Multiple Implementation

Bước 1: Tạo Interface

Tạo file: `IPlayable.java`

```java
package com.bootcamp.lab3;

public interface IPlayable {
    // Interface chỉ định nghĩa method, không có body
    void play();

    // Java 8+: Default method (có implementation)
    default void stop() {
        System.out.println("Đã dừng");
    }
}
```

Giải thích:

- `interface`: Hợp đồng hành vi
- Class implement interface phải implement tất cả methods
- `default method`: Có implementation, không bắt buộc override

Bước 2: Implement Interface

Tạo file: `Guitar.java`

```java
package com.bootcamp.lab3;

public class Guitar implements IPlayable {
    @Override
    public void play() {
        System.out.println("🎸 Tưng tưng tưng...");
    }
}
```

Tạo file: `VideoGame.java`

```java
package com.bootcamp.lab3;

public class VideoGame implements IPlayable {
    private String gameName;

    public VideoGame(String gameName) {
        this.gameName = gameName;
    }

    @Override
    public void play() {
        System.out.println("🎮 " + gameName + ": Pew pew pew...");
```

```
    }
}
```

Bước 3: Test Interface

Tạo file: `PlayableTest.java`

```java
package com.bootcamp.lab3;

public class PlayableTest {
    public static void main(String[] args) {
        // Tạo mảng IPlayable
        IPlayable[] devices = new IPlayable[3];

        devices[0] = new Guitar();
        devices[1] = new VideoGame("Super Mario");
        devices[2] = new Guitar();

        System.out.println("=== CHƠI ĐỒ CHƠI ===\n");

        // POLYMORPHISM với Interface
        for (IPlayable device : devices) {
            device.play();
        }

        // Test method nhận Interface
        testDevice(new Guitar());
        testDevice(new VideoGame("Pacman"));
    }

    // Method nhận Interface - Linh hoạt!
    public static void testDevice(IPlayable device) {
        System.out.println("\n--- Testing device ---");
        device.play();
        device.stop();  // Default method
    }
}
```

Giải thích:

- Interface cho phép **Multiple Implementation**
- Class không liên quan (Guitar, VideoGame) có thể cùng implement một interface
- Method nhận Interface → Có thể nhận bất kỳ class nào implement interface đó

📝 PHẦN 2: BÀI TẬP CÓ HƯỚNG DẪN

Bài 3.1: Hình Học (Abstract Class)

Đề bài: Tạo hệ thống hình học với Abstract Class

Yêu Cầu

1. **Abstract Class `Shape`:**
   - Attributes: `color` (String)
   - Abstract methods: `calculateArea()`, `calculatePerimeter()`
   - Concrete method: `displayInfo()`
2. **Class `Circle` extends `Shape`:**
   - Attribute: `radius`
   - Implement abstract methods
3. **Class `Rectangle` extends `Shape`:**
   - Attributes: `width`, `height`
   - Implement abstract methods

Hướng Dẫn

Bước 1: Abstract Class

```java
package com.bootcamp.lab3;

public abstract class Shape {
    protected String color;

    public Shape(String color) {
        this.color = color;
    }

    public abstract double calculateArea();
    public abstract double calculatePerimeter();

    public void displayInfo() {
        System.out.println("=== HÌNH HỌC ===");
        System.out.println("Màu: " + color);
        System.out.printf("Diện tích: %.2f\n", calculateArea());
        System.out.printf("Chu vi: %.2f\n", calculatePerimeter());
        System.out.println("===============\n");
    }

    public String getColor() {
        return color;
    }
}
```

Bước 2: Circle

```java
package com.bootcamp.lab3;

public class Circle extends Shape {
    private double radius;
    private static final double PI = 3.14159;

    public Circle(String color, double radius) {
        super(color);
        this.radius = radius;
    }

    @Override
    public double calculateArea() {
        return PI * radius * radius;
    }

    @Override
    public double calculatePerimeter() {
        return 2 * PI * radius;
    }

    public double getRadius() {
        return radius;
    }
}
```

Bước 3: Rectangle

```java
package com.bootcamp.lab3;

public class Rectangle extends Shape {
    private double width;
    private double height;

    public Rectangle(String color, double width, double height) {
        super(color);
        this.width = width;
        this.height = height;
    }

    @Override
    public double calculateArea() {
        return width * height;
    }

    @Override
    public double calculatePerimeter() {
        return 2 * (width + height);
    }

    public double getWidth() { return width; }
```

```
    public double getHeight() { return height; }
}
```

Bước 4: Test

```
package com.bootcamp.lab3;

public class ShapeTest {
    public static void main(String[] args) {
        // Mảng Shape (kiểu cha)
        Shape[] shapes = new Shape[3];

        shapes[0] = new Circle("Đỏ", 5.0);
        shapes[1] = new Rectangle("Xanh", 4.0, 6.0);
        shapes[2] = new Circle("Vàng", 3.0);

        // POLYMORPHISM: Tính tổng diện tích
        double totalArea = 0;
        for (Shape shape : shapes) {
            shape.displayInfo();
            totalArea += shape.calculateArea();
        }

        System.out.printf("Tổng diện tích: %.2f\n", totalArea);
    }
}
```

Bài 3.2: Loại Bỏ If/Else bằng Polymorphism

Đề bài: Refactor code có nhiều if/else thành Polymorphism

Code Cũ (Có If/Else)

```
public void processAnimal(Animal animal) {
    if (animal instanceof Dog) {
        Dog dog = (Dog) animal;
        dog.bark();
        dog.fetch();
    } else if (animal instanceof Cat) {
        Cat cat = (Cat) animal;
        cat.meow();
        cat.scratch();
    } else if (animal instanceof Bird) {
        Bird bird = (Bird) animal;
        bird.tweet();
        bird.fly();
    }
}
```

Code Mới (Polymorphism)

Bước 1: Thêm Abstract Method vào Animal

```
public abstract class Animal {
    // ... existing code ...

    // Abstract method - Mỗi subclass implement khác
    public abstract void performAction();
}
```

Bước 2: Implement ở Subclass

```
public class Dog extends Animal {
    // ... existing code ...

    @Override
    public void performAction() {
        bark();
        fetch();
    }
}

public class Cat extends Animal {
    // ... existing code ...

    @Override
    public void performAction() {
        meow();
        scratch();
    }
}
```

Bước 3: Code Xử Lý Đơn Giản

```
public void processAnimal(Animal animal) {
    animal.makeSound();
    animal.performAction();  // Polymorphism tự động chọn đúng method
}
```

Lợi ích:

- ✅ Không cần if/else
- ✅ Dễ mở rộng (thêm class mới, không sửa code cũ)
- ✅ Tuân thủ Open/Closed Principle

🎯 PHẦN 3: BÀI TẬP TỰ LUYỆN (KHÔNG CÓ HƯỚNG DẪN)

Bài 3.1: Class Vehicle và Subclasses

Đề bài: Tạo hệ thống phương tiện giao thông:

1. **Class `Vehicle` (superclass):**
   - Attributes: `brand`, `year`
   - Methods: `start()`, `stop()`, `displayInfo()`
2. **Class `Car` extends `Vehicle`:**
   - Attribute: `numberOfDoors`
   - Override `displayInfo()`
3. **Class `Motorcycle` extends `Vehicle`:**
   - Attribute: `hasSidecar` (boolean)
   - Override `displayInfo()`

Yêu cầu:

- Sử dụng constructor chaining
- Test với mảng `Vehicle[]`

Bài 3.2: Interface Drawable và Resizable

Đề bài: Tạo 2 interfaces:

1. **Interface `Drawable`:**
   - Method: `draw()`
2. **Interface `Resizable`:**
   - Method: `resize(double factor)`
3. **Class `Circle` implement cả 2:**
   - Attribute: `radius`
   - Implement cả 2 methods

Yêu cầu:

- Test với method nhận Interface

Bài 3.3: Abstract Class Employee với Bonus

Đề bài: Mở rộng LAB 3.2:

1. Thêm abstract method `calculateBonus()`
2. `Manager`: Bonus = 20% lương
3. `Worker`: Bonus = 10% lương
4. `SalesPerson`: Bonus = 15% lương + commission

Yêu cầu:

- Tạo class `SalesPerson` mới
- Tính tổng bonus của tất cả nhân viên

Bài 3.4: Class Shape với Abstract Methods

Đề bài: Tạo hệ thống hình học:

1. **Abstract class `Shape`:**
   - Attributes: `color` (String), `filled` (boolean)
   - Abstract methods: `getArea()`, `getPerimeter()`
   - Concrete method: `displayInfo()`
2. **Class `Circle` extends `Shape`:**
   - Attribute: `radius`
   - Implement abstract methods
3. **Class `Rectangle` extends `Shape`:**
   - Attributes: `width`, `height`
   - Implement abstract methods

Gợi ý:

- Circle: Area = πr^2, Perimeter = $2\pi r$
- Rectangle: Area = width × height, Perimeter = 2(width + height)
- Dùng mảng `Shape[]` để test polymorphism

Bài 3.5: Interface Comparable và Sorting

Đề bài: Tạo class `Student` implement `Comparable<Student>`:

- Attributes: `name`, `age`, `gpa`
- Implement `compareTo()`: So sánh theo GPA (cao → thấp)
- Nếu GPA bằng nhau, so sánh theo tên (A-Z)

Gợi ý:

- `compareTo()` trả về: số âm (nhỏ hơn), 0 (bằng), số dương (lớn hơn)
- Dùng `String.compareTo()` để so sánh tên
- Tạo mảng Student, sắp xếp bằng `Arrays.sort()`

Yêu cầu:

- Hiển thị danh sách trước và sau khi sắp xếp

Bài 3.6: Multi-level Inheritance (Kế Thừa Nhiều Cấp)

Đề bài: Tạo hệ thống động vật 3 cấp:

1. **Class `Animal` (base):**
   - Attributes: `name`, `age`
   - Methods: `eat()`, `sleep()`
2. **Class `Mammal` extends `Animal`:**
   - Attribute: `furColor`
   - Override `eat()`: "Động vật có vú đang ăn..."
3. **Class `Dog` extends `Mammal`:**
   - Attribute: `breed`
   - Override `eat()`: "Chó đang ăn..."
   - Method: `bark()`

Gợi ý:

- Constructor chaining: Dog → Mammal → Animal
- Test với `Animal a = new Dog()` và gọi `eat()`
- Quan sát polymorphism hoạt động như thế nào

Bài 3.7: Interface Flyable và Swimmable

Đề bài: Tạo 2 interfaces:

1. **Interface `Flyable`:**
   - Method: `fly()`, `getMaxAltitude()`
2. **Interface `Swimmable`:**
   - Method: `swim()`, `getMaxDepth()`
3. **Classes:**
   - `Bird` implements `Flyable`
   - `Fish` implements `Swimmable`
   - `Duck` implements cả 2 (vịt vừa bay vừa bơi)

Gợi ý:

- Duck phải implement cả 4 methods
- Tạo method nhận `Flyable` parameter để test
- Tạo method nhận `Swimmable` parameter để test

Bài 3.8: Abstract Class với Template Method Pattern

Đề bài: Tạo abstract class `Beverage` (đồ uống):

- Template method `prepare()` (final):
  1. Boil water
  2. Brew (abstract - mỗi loại khác nhau)
  3. Pour in cup
  4. Add condiments (abstract - mỗi loại khác nhau)
- Classes:
  - `Coffee` extends `Beverage`
  - `Tea` extends `Beverage`

Gợi ý:

- Template method định nghĩa thuật toán chung
- Các bước cụ thể để subclass implement
- Đảm bảo quy trình nhất quán

🏆 PHẦN 4: THỬ THÁCH NÂNG CAO

Challenge 1: RPG Game (Game Nhập Vai)

Đề bài: Xây dựng hệ thống chiến đấu Turn-based.

Yêu cầu:

1. **Abstract Class `Character`:**
   - Attributes: `hp` (máu), `name`, `damage` (sát thương)
   - Abstract method: `attack(Character target)`
   - Concrete methods:
     - `takeDamage(int amount)`: Trừ máu
     - `isAlive()`: Trả về true nếu hp > 0
2. **Class `Warrior` extends `Character`:**
   - HP cao, damage thấp
   - `attack()`: 20% cơ hội đánh đôi (X2 damage)
3. **Class `Mage` extends `Character`:**
   - HP thấp, damage cao
   - `attack()`: Tốn 1 lượt để gồng năng lượng, lượt sau tung chưởng X3 damage
4. **Main Game Loop:**
   - Tạo Warrior vs Mage
   - Vòng lặp `while` cho đến khi một người chết

    - In ra người thắng

Gợi ý:

- Dùng `Math.random()` để tính xác suất
- Dùng biến boolean để đánh dấu trạng thái "đang gồng"

Challenge 2: Payment System với Interface

Đề bài: Tạo hệ thống thanh toán đa kênh.

Yêu cầu:

1. **Interface `PaymentMethod`:**
    - Method: `pay(double amount)`
2. **Classes implement:**
    - `CreditCard`: Pay với credit card
    - `PayPal`: Pay với PayPal
    - `Momo`: Pay với Momo
3. **Class `PaymentService`:**
    - Method: `processPayment(PaymentMethod method, double amount)`
    - Sử dụng Polymorphism
4. **Factory Pattern:**
    - Class `PaymentFactory`:
        - Method: `createPaymentMethod(String type)`
        - Trả về PaymentMethod tương ứng

Yêu cầu:

- Loại bỏ hoàn toàn if/else trong code xử lý
- Dễ thêm phương thức thanh toán mới

📚 BẢNG GHI NHỚ (CHEAT SHEET)

Inheritance

Cú pháp	Ý nghĩa	Ví dụ
`extends`	Kế thừa class	`class Dog extends Animal`
`super()`	Gọi constructor cha	`super(name)`
`super.method()`	Gọi method cha	`super.displayInfo()`
`@Override`	Đánh dấu override	Annotation

Abstract vs Interface

Đặc điểm	Abstract Class	Interface
Từ khóa	`abstract class`	`interface`
Fields	Có thể có	Chỉ constants
Methods	Abstract + Concrete	Chỉ abstract (trước Java 8)
Constructor	Có	Không
Multiple	Chỉ 1 class	Nhiều interfaces
Khi nào dùng	Quan hệ IS-A	Hành vi CAN-DO

Polymorphism

Khái niệm	Ý nghĩa	Ví dụ
Upcasting	Con → Cha (tự động)	`Animal a = new Dog()`
Downcasting	Cha → Con (phải cast)	`Dog d = (Dog) a`
instanceof	Kiểm tra kiểu	`if (a instanceof Dog)`
Dynamic Binding	Chọn method tại runtime	Polymorphism

✅ CHECKLIST HOÀN THÀNH

- ☐ Đã hiểu kế thừa và `extends`
- ☐ Đã sử dụng `super()` và `super.method()`
- ☐ Đã viết override method với `@Override`
- ☐ Đã phân biệt Overriding vs Overloading
- ☐ Đã tạo Abstract Class
- ☐ Đã implement Interface
- ☐ Đã áp dụng Polymorphism
- ☐ Đã hiểu Upcasting/Downcasting
- ☐ Đã hoàn thành LAB 3.1 (Zoo)
- ☐ Đã hoàn thành LAB 3.2 (Employee)
- ☐ Đã hoàn thành LAB 3.3 (IPlayable)
- ☐ Đã làm ít nhất 2 bài tập có hướng dẫn
- ☐ Đã làm ít nhất 2 bài tập tự luyện
- ☐ Đã thử ít nhất 1 thử thách nâng cao

🎓 TỔNG KẾT

Sau bài thực hành này, bạn đã:

1. ✅ **Sử dụng kế thừa** để tái sử dụng code
2. ✅ **Hiểu Overriding** và `@Override`

3. ✅ **Tạo Abstract Class** và Interface
4. ✅ **Áp dụng Polymorphism** để loại bỏ if/else
5. ✅ **Hiểu Upcasting/Downcasting**
6. ✅ **Tư duy OOP** nâng cao

Bước tiếp theo: Chuyển sang Thực hành 4: Nguyên lý thiết kế & Design Patterns

---

Chúc bạn học tốt! 🚀